# Light Chase

Android Project Outline | Entertainment / Action strategy | Android Studio + Java

**Submission status.** Research and proposed design; no gameplay implementation or playtest results are claimed. Team names, repository URL, current teaching week and submission language must be confirmed before submission. English is a working assumption. The week 6-15 schedule on page 4 is conditional.

## App idea and intended users

Light Chase is a short, two-player, top-down stealth game for Android. Friends on the same local network take opposite roles: a police officer searches with a directional flashlight, while a thief transports three parcels between paired stations and then escapes. Walls stay visible, but opponent information depends on each role. The intended benefit is repeatable tactical play through route choice, aim, bluffing and role swapping.

The first release targets one fixed map, one police officer and one thief, and a 180-second round. It requires two Android phones and a laptop running a Java server. A laptop-free host, public matchmaking, accounts, ranked play and procedural levels are outside the committed scope. Entertainment value remains a playtest hypothesis.

## Related apps and the design opportunity

| Reference | Verified functions and relevant components | Proposed development |
|---|---|---|
| Robbery Bob [1] Android | Single-player stealth puzzles; loot collection; guards, hiding and distinct maps. Store images show a top-down play area and touch controls. | Replace a scripted guard challenge with a second human player who searches through constrained visibility. |
| Identity V [2] Android | Asymmetric 1v4 play: survivors decode machines and escape while a hunter pursues; selectable characters. | Focus on readable 1v1, top-down flashlight tracking and moving cargo rather than a broad character roster. |
| Burgle Bros [3] Android | Team heist planning, guards, character powers, floors and tools such as smoke and EMP. | Adapt the route-planning appeal to a compact real-time duel with paired pickup and delivery objectives. |

**Novelty boundary.** Invisigun Reloaded [4] already uses invisible multiplayer characters and environmental clues. Stealth, asymmetry, smoke and invisibility are established ideas. Our proposed contribution is their specific integration: sustained-light marking, temporary post-mark escape, and cargo routing that creates competing search priorities. This is not a claim of worldwide originality. Competitor comparisons are based on publisher listings, not hands-on tests.

## Chosen implementation base

Build a new game on the open-source libGDX framework [5], using Java-WebSocket [6] for transport. The commercial reference games provide design comparisons only. Their listed pages do not provide a reusable game codebase; their images are not proposed shipping assets. Our rules, map, state machine, visibility filtering and evaluation will be original project work.

# Planned features and interaction

All numerical values below are starting hypotheses to be adjusted after testing.

| Feature | Proposed behaviour and implementation |
|---|---|
| F1 Two-role match | Room join, role selection and ready state; server starts a shared timer. Thief delivers 3 parcels, carrying only 1 at a time, then channels at either of 2 exits. Police wins at 3 marks or timeout. |
| F2 Directional detection | A target is detectable only if it is within the flashlight range and angle, with no wall or smoke blocking line of sight. A continuous 0.8 s detection gives 1 mark; interruption resets progress. Server uses cone geometry and grid raycasts. |
| F3 Escape opportunity | The first 2 marks trigger 1.5 s true invisibility, followed by 1 s protection from another mark. The third ends the match immediately. Invisibility hides the body and cargo and blocks pickup, delivery and exit actions. |
| F4 Route decisions | Three fixed pickup/destination pairs; 1 s stationary interactions, cancelled by movement or a mark. Delivery broadcasts only a coarse map zone and completed count. Hidden inventory and remote parcel changes are withheld from the police. |
| F5 Two counter-tools | Police wide beam: wider but shorter light, proposed 3 s duration / 15 s cooldown. Thief smoke: local opaque area, proposed 3 s / 20 s. Server owns activation and cooldowns. Both are added after the basic duel works. |
| F6 Results and settings | Local results, event timeline and role filters; no full replay. Store control size, sensitivity, sound and vibration settings. Users can clear history. Offline menus and a guided rules tutorial remain usable without a server. |

## Screen flow and accessibility

Home leads to a rules tutorial, join-room screen, history or settings. The lobby shows connection status, occupied roles and ready state. Gameplay uses a left movement pad; police uses a right aiming pad, while thief uses an interaction button. Results explain the win condition and offer role-swapped rematch. Disconnection ends the round as interrupted with a clear reason; backgrounding cancels active input and sends a leave signal when possible.

Use role names and shapes as well as blue/amber colours, scalable touch controls, high-contrast HUD text and optional vibration. Avoid relying on sound alone. Prevent wall and edge clutter from covering controls. Menus follow familiar Android navigation; gameplay needs custom rendering. The slide deck includes an AI-generated proposed gameplay view and a complete screen/state inventory.

## Scope and consistency rules

Mandatory: F1-F4, F6, one map and two-device LAN play. Planned extension within the baseline: F5. Optional only after acceptance: bot practice, extra maps or decoy footsteps. A server tick resolves timeout before the third mark, then successful escape. Characters do not body-block each other. One authoritative result is recorded per match. These choices remove ambiguous ties and accidental instant recapture.

# Technical approach and reuse

Reuse infrastructure; implement and explain the game-specific logic.

| Source / tool | Reuse and integration | Project-owned work |
|---|---|---|
| libGDX [5] Apache-2.0 code | SpriteBatch, camera, input, Screen lifecycle and tile-map support [7]; Java client in Android Studio. | Touch control mapping, map layout, UI flow, collision and visibility rules. Art is licensed separately. |
| Java-WebSocket [6] MIT | Java server/client transport; study ChatServer example for callbacks and connection lifecycle. | Room protocol, input validation, fixed-tick simulation, role-filtered snapshots and disconnect policy. |
| Room 2.x [8] AndroidX | Java annotationProcessor; local SQLite abstraction in Android module; asynchronous repository calls. | Match/result entities, event retention, history queries and deletion. Core game has no Room dependency. |
| Optional gdx-ai [9] Apache-2.0 | Pathfinding / state machines only if bot practice is later approved within time. | Bot perception must use the same limited observations; no hidden-position access. |

Keep a dependency and asset register with source URL, chosen version/commit, licence and modification notes. Preserve applicable notices in a THIRD_PARTY_NOTICES file. Pin versions only after the Android build and LAN spike succeed. No source library has yet been integrated or compatibility-tested in this project. [5][6][8][9]

## Architecture and network model

Use separate Java modules for shared simulation, libGDX client, Android services and a desktop Java server. Clients send sequenced movement, aim and action requests. The server validates speed, collisions, cooldowns and objectives, then sends only permitted observations to each role. Start with a 30 Hz simulation and 10-15 Hz snapshots; render up to the device target rate with interpolation. These rates are design targets, not measured performance.

When an opponent becomes hidden, clear its render/interpolation cache immediately. Do not send hidden coordinates, cargo status or precise remote pickup events. Network callbacks enqueue immutable messages; rendering runs on the game thread [10]. Begin with manual server address and a room code. Verify LAN routing, Android permissions [11], background handling and network security configuration in the first spike; a room code alone is not authentication.

## Input, processing, storage and output

| Area | Evidence planned for the four-aspect rubric |
|---|---|
| Input | Multi-touch controls, aim, action, room address/code and settings; validate malformed or repeated requests. |
| Processing | Fixed-step simulation; map collision; cone and raycast checks; mark/cloak states; delivery and victory logic. |
| Storage | Authoritative live state in server memory; Room stores local MatchResult and MatchEvent records, with match ID deduplication and clear-history support. Never write every rendered frame to storage. |
| Output / integration | Role-specific map, progress and cooldown feedback; sound/haptics; results and filtered history; observable error states from connection to storage. |

# Work plan and evaluation

Conditional ten-week plan: assumes work starts in teaching week 6; confirm before submission.

| Week | Work and milestone evidence |
|---|---|
| 6 | Freeze rules and map sketch; prove two phones can exchange inputs via Java server. Record dependency versions and device / network details. |
| 7 | Movement, collision, aiming and wall-occluded vision; tests for walls, corners, cone edges and equal diagonal speed. |
| 8 | Mark / cloak state machine, pickup / delivery / exits, timer and tie rules. Local playable loop and test log. |
| 9 | Integrate authoritative LAN match and per-role filtering. Alpha target: complete two-device round, known-issue list and source/learning log. |
| 10 | Add wide beam and smoke; cooldown validation; early role-swapped playtests and map tuning. |
| 11 | Room results/events, history deletion, tutorial, settings and lifecycle/error states. |
| 12 | Beta target: feature-complete scope, device checks, disconnect tests and frozen feature list. |
| 13 | 20-30 exploratory role-swapped rounds across several players; log win reasons, duration, marks and unclear feedback. Tune, then recheck affected rules. |
| 14 | Fix regressions; prepare source archive, APK, MP4 demonstration, contribution evidence, updated report and slides. |
| 15 | Final acceptance on two phones; submit by the actual course deadline and prepare individual explanation of code and learning. |

Course document 00 gives approximate Alpha weeks 8-9, Beta 11-12 and Final 15-16. The pasted Outline instruction explicitly ends at week 15, so this plan targets week 15; confirmed Moodle dates take priority. If the actual start is later than week 6, recalculate capacity and remove optional work first. Team roles are proposed workstreams (simulation/network; Android/UI/storage; QA/documentation), not assigned people. Estimated total: 160-220 person-hours, subject to the initial spike.

## Acceptance and learning evidence

Before calling the baseline complete: finish a full match on two physical phones; prove walls and invisibility prevent marking; inspect snapshots for hidden-data leakage; reject impossible inputs and duplicate deliveries; verify identical winner on both clients; restore saved results after restart and delete them; show a clear disconnect result without a crash. Measure frame time and latency on named devices rather than promising 60 fps. Playtests explore balance, not statistical proof of fairness. Keep commits, issue logs and Alpha/Beta/Final reflections on Java architecture, Android lifecycle, persistence and networking.

## References

[1] Robbery Bob | [2] Identity V | [3] Burgle Bros | [4] Invisigun Reloaded | [5] libGDX source and licence | [6] Java-WebSocket source and examples | [7] libGDX tile maps | [8] Room Java configuration | [9] gdx-ai | [10] libGDX threading | [11] Android LAN permissions

Course basis: supplied documents 00 (scope, attribution, learning and approximate submissions), 01 (four aspects), 02 (Outline requirements), 03 (exemplar structure) and 04 (Outline rubric); plus the pasted Moodle instructions. Source access date: 7 October 2026. The accompanying research report contains the full requirement crosswalk and source register.